Assignment2

January 30, 2026

DSCI372 Elisa Herter

[38]:
```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
```

#Q1

[39]:
```
def sign(z):
    return 1 if z >= 0 else -1

def check_predictions(X, y, w):
    preds = np.array(X.dot(w))
    for i in range(len(preds)):
        preds[i] = 1 if preds[i] >= 0 else -1
    return preds == y

df_perceptron = pd.DataFrame({
    "x1": [1, 2, -3, -3],
    "x2": [1, -1, -1, 1],
    "y":  [1, -1, 1, 1]  # y is already in {-1, 1}
})

X = df_perceptron[["x1", "x2"]].values
y = df_perceptron["y"].values

# (a) Train in order i=1,2,3,4
print("Part (a) Training in order i=1,2,3,4:")
w = np.array([0, 0])  # initialize weights

for i, (xi, yi) in enumerate(zip(X, y), start=1):
    pred_val = np.dot(w, xi)
    pred = sign(pred_val)

    # Update weights if mistake occurs
    if pred != yi:
        print(f"Iteration {i}: Mistake made. Weights before update: {w}")
```

```
        w = w + yi * xi
    else:
        print(f"Iteration {i}: Correct classification. Weights unchanged: {w}")

    print(f"Weights after iteration {i}: {w}\n")

correct = check_predictions(X, y, w)
print(f"After training, perceptron correctly classifies all points: {correct.
 ↪all()}\n")
```

```
Part (a) Training in order i=1,2,3,4:
Iteration 1: Correct classification. Weights unchanged: [0 0]
Weights after iteration 1: [0 0]

Iteration 2: Mistake made. Weights before update: [0 0]
Weights after iteration 2: [-2  1]

Iteration 3: Correct classification. Weights unchanged: [-2  1]
Weights after iteration 3: [-2  1]

Iteration 4: Correct classification. Weights unchanged: [-2  1]
Weights after iteration 4: [-2  1]

After training, perceptron correctly classifies all points: False
```

[40]:
```
# (b) Train in order i=2,1,3,4
w = np.array([0, 0])  # reset weights
order = [1, 0, 2, 3]  # FOR i=2,1,3,4

for idx in order:
    xi, yi = X[idx], y[idx]
    pred_val = np.dot(w, xi)
    pred = sign(pred_val)

    if pred != yi:
        print(f"Iteration {idx+1}: Mistake made. Weights before update: {w}")
        w = w + yi * xi
    else:
        print(f"Iteration {idx+1}: Correct classification. Weights unchanged:␣
 ↪{w}")

    print(f"Weights after iteration {idx+1}: {w}\n")

correct = check_predictions(X, y, w)
print(f"After training, perceptron correctly classifies all points: {correct.
 ↪all()}")
```

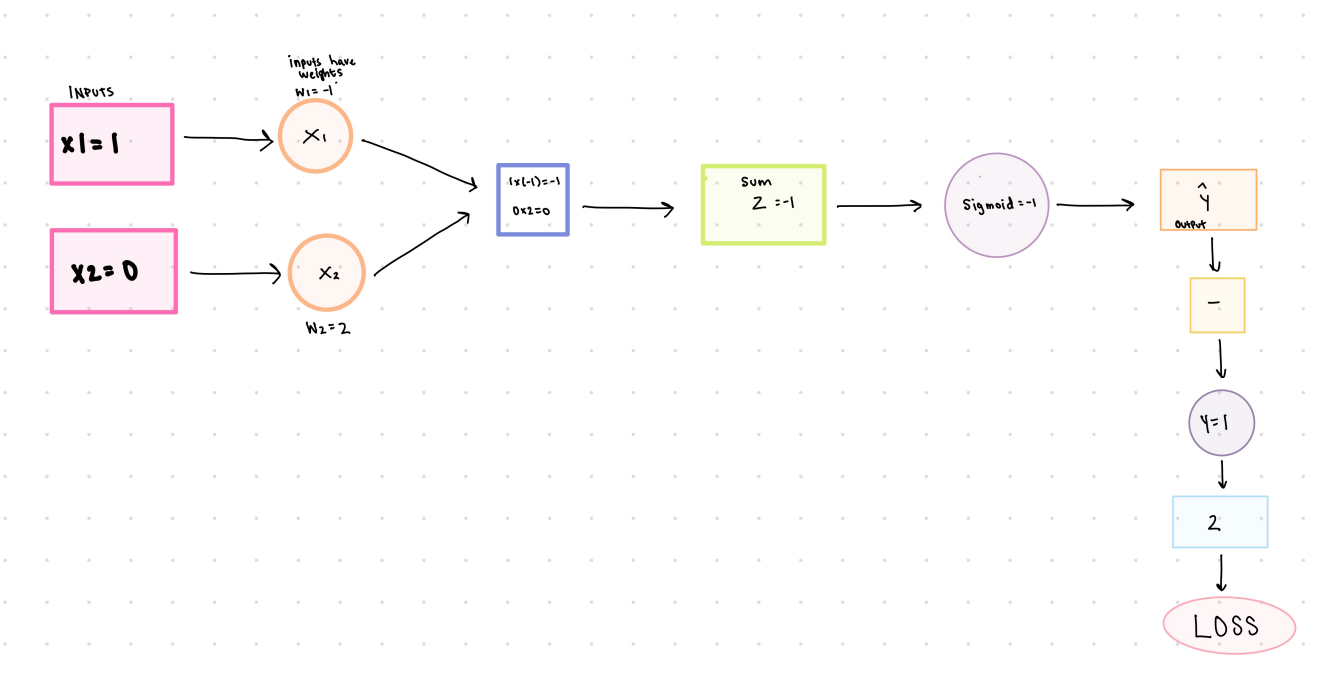
```
Iteration 2: Mistake made. Weights before update: [0 0]
Weights after iteration 2: [-2  1]

Iteration 1: Mistake made. Weights before update: [-2  1]
Weights after iteration 1: [-1  2]

Iteration 3: Correct classification. Weights unchanged: [-1  2]
Weights after iteration 3: [-1  2]

Iteration 4: Correct classification. Weights unchanged: [-1  2]
Weights after iteration 4: [-1  2]

After training, perceptron correctly classifies all points: True
```

#Q2

[41]:
```
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Given values
x = np.array([1, 0])        # input vector x = [1, 0]
y = 1                       # true label y = 1
w = np.array([-1, 2])       # weights w = [-1, 2]

# ===== Forward pass =====
print("=== Forward pass ===")

z = np.dot(w, x)
print(f"z = w · x = (-1)(1) + 2(0) = {z}")

sigma_val = sigmoid(z)
print(f"ŷ = σ(z) = 1 / (1 + e^(-z)) = 1 / (1 + e^{ -z if z < 0 else z }) ␣
↪{sigma_val:.9f}")

pred_loss = (sigma_val - y)**2
print(f"prediction loss = (σ(z) - y)^2 = ({sigma_val:.9f} - {y})^2 = {pred_loss:.
↪9f}")
print()

# ===== Backward pass =====
print("=== Backward pass ===")

dloss_dsigma = 2 * (sigma_val - y)
print(f"loss/ŷ = 2(ŷ - y) = 2({sigma_val:.9f} - {y}) = {dloss_dsigma:.9f}")

dsigma_dz = sigma_val * (1 - sigma_val)
print(f"σ/z = σ(z)(1 - σ(z)) = {sigma_val:.9f} × (1 - {sigma_val:.9f}) =␣
↪{dsigma_dz:.9f}")
```

```
dloss_dz = dloss_dsigma * dsigma_dz
print(f"loss/z = loss/ŷ × σ/z = {dloss_dsigma:.9f} × {dsigma_dz:.9f} = {dloss_dz:
 ↪.9f}")

dloss_dw = dloss_dz * x
dloss_dw1, dloss_dw2 = dloss_dw[0], dloss_dw[1]
print(f"loss/w_1 = loss/z × x_1 = {dloss_dz:.9f} × {x[0]} = {dloss_dw1:.9f}")
print(f"loss/w_2 = loss/z × x_2 = {dloss_dz:.9f} × {x[1]} = {dloss_dw2:.9f}")
print(f"loss/w = [{dloss_dw1:.3f}, {dloss_dw2:.3f}]")
```

```
=== Forward pass ===
z = w · x = (-1)(1) + 2(0) = -1
ŷ = σ(z) = 1 / (1 + e^(-z)) = 1 / (1 + e^1)  0.268941421
prediction loss = (σ(z) - y)^2 = (0.268941421 - 1)^2 = 0.534446645

=== Backward pass ===
loss/ŷ = 2(ŷ - y) = 2(0.268941421 - 1) = -1.462117157
σ/z = σ(z)(1 - σ(z)) = 0.268941421 × (1 - 0.268941421) = 0.196611933
loss/z = loss/ŷ × σ/z = -1.462117157 × 0.196611933 = -0.287469681
loss/w_1 = loss/z × x_1 = -0.287469681 × 1 = -0.287469681
loss/w_2 = loss/z × x_2 = -0.287469681 × 0 = -0.000000000
loss/w = [-0.287, -0.000]
```



- The yellow box labeled sum is a z not a 2

#Q3

(a)

If $z > 0$, then $\phi(z) = z$ and

$$\frac{\partial \phi(z)}{\partial z} = 1.$$

If $z < 0$, then $\phi(z) = 0$ and

$$\frac{\partial \phi(z)}{\partial z} = 0.$$

$$\frac{\partial \phi(z)}{\partial z} = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y,$$

$$\frac{\partial \hat{y}}{\partial z} = \frac{\partial \phi(z)}{\partial z},$$

$$\frac{\partial z}{\partial w} = x.$$

$$\frac{\partial L}{\partial w} = (\hat{y} - y)\, \frac{\partial \phi(z)}{\partial z}\, x.$$

Or

$$\frac{\partial L}{\partial w} = \begin{cases} (\hat{y} - y)\, x, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

(c)

We have

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot \frac{\partial \phi(z)}{\partial z} \cdot x \qquad \text{with} \qquad \frac{\partial \phi(z)}{\partial z} = \begin{cases} 1 & z > 0 \\ 0 & z \leq 0 \end{cases}$$

Thus

$$\frac{\partial L}{\partial w} = \begin{cases} (\hat{y} - y)\, x & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Require $\frac{\partial L}{\partial w} = 0 \quad \text{and} \quad \hat{y} \neq y$.

This holds when

1. $z \leq 0 \quad \text{and} \quad x \neq 0 \quad \text{and} \quad \neq y$
2. $x = 0 \quad \text{and} \quad \neq y$

$\boxed{wx + b \leq 0, \quad x \neq 0, \quad \max(0, wx + b) \neq y}$ Or $\boxed{\text{z}\leq 0,\ x \neq 0,\ \hat{y} \neq y)}$

(d)

$$z \leq 0 \;\Rightarrow\; \frac{\partial \phi(z)}{\partial z} = 0 \;\Rightarrow\; \frac{\partial L}{\partial w} = 0$$

Gradient descent will not update w because $\frac{\partial L}{\partial w} = 0$, which occurs due to the ReLU activation being zero, when $z \leq 0$.

(e)

The leaky ReLU is much less likely to stop learning because its derivative is non-zero everywhere:

$$\frac{\partial \phi}{\partial z} = \begin{cases} \alpha & \text{if } z < 0, \\ 1 & \text{if } z \geq 0, \end{cases}$$

with $\alpha > 0$. This ensures a non-zero gradient flows through the activation for all z, so $\frac{\partial L}{\partial w}$ remains non-zero (assuming $x \neq 0$ and $\hat{y} \neq y$), allowing w to receive updates even when the pre-activation z is negative.